

SNA4DS cheatsheet

Automatically generated from the current snafun vignette tables and figures.

Contents

- [Main packages in the SNA4DS course](#)
 - [Overview of igraph and network graph objects](#)
 - [The snafun package](#)
- [Creating a graph object](#)
 - [Additional graph creation functions](#)
- [Converting between graph classes](#)
- [Manipulating the graph object](#)
- [Graph level indices](#)
 - [Communities and other subgroups](#)
- [Vertex-level indices](#)
- [Dyad-level indices](#)
- [Plotting](#)
 - [Plotting in snafun](#)
- [Statistical models](#)
 - [Overview table](#)
 - [Network autocorrelation models](#)
 - [Conditional Uniform graphs \(CUG\)](#)
 - [QAP test](#)
 - [QAP linear regression](#)
- [Temporal networks \(exploration and description\)](#)
 - [Network generation and manipulation](#)
 - [QAP logistic regression](#)
 - [Terms classification for every Exponential Random Graph Model \(ERGM\)](#)
 - [Bipartite ERGMs](#)
 - [ERGM for temporal networks](#)
 - [Network measures and descriptives](#)
 - [Participation shifts](#)

In this *cheatsheet* we summarize the main R functions that are used in the SNA4DS course. We will not explain the underlying concepts here, but refer you to the lectures, labs, and slides of the course for that.

The aim of this cheatsheet is that it provides you with an overview of the main functions you will need throughout the course. We hope that it can provide a useful reference for you, as you develop and apply your network analysis skills.

NOTE:

Most functions have multiple arguments. Our aim is in this cheatsheet not to show and discuss the various arguments that exist, because that would yield an unwieldy and very long document. Rather, we recommend you use your R skills and use the help function `?` and `help` and other approaches we teach you in this course to learn about the details of a specific function. If you still can't figure it out, contact us and we'll assist you.

Main packages in the SNA4DS course

Overview of igraph and network graph objects

There are two main packages for basic graph generation and manipulation: the `igraph` package and the `statnet` package. Actually, `statnet` is a suite of packages that work together. In this course, we will make use of several packages from the `statnet` suite.

The `igraph` package creates a graph object of type `igraph`. The `statnet` suite creates a graph object of type `network`. Both can generate graphs and do basic manipulation. The `igraph` package provides more mathematical graph functions, while the `statnet` suite provides many statistical models.

The snafun package

The `igraph` package and `statnet` suite are jointly very powerful, but they each require graph objects with specific structures. If you want to combine functions from `igraph`, `network`, and `sna`, you often end up converting between `igraph`, `network`, `matrix`, and `edgelist` representations.

Believe it or not, this is a pain and quite annoying.

THE snafun PACKAGE TO THE RESCUE!

- It provides a fairly consistent API, so you don't have to constantly re-learn different argument conventions.
- Most functions work on `igraph`, `network`, matrices, and edgelists/data frames.
- It lets you focus on the fun of network analysis rather than the friction of conversions and mismatched interfaces.

Creating a graph object

Graph creation

CREATE AN EMPTY NETWORK

snafun	snafun::create_empty_graph(n_vertices, directed = TRUE, graph = c("igraph", "network"))
--------	---

igraph	igraph::make_empty_graph()
--------	----------------------------

network	network::network::initialize()
---------	--------------------------------

CREATE A MANUAL (LITERAL) NETWORK

snafun	snafun::create_manual_graph(A -+ B -+ C, simplify = TRUE, graph = 'igraph')
--------	---

igraph	igraph::graph_from_literal(A -+ B -+ C, simplify = TRUE)
--------	--

network	-
---------	---

CREATE A RING NETWORK

snafun	-
--------	---

igraph	igraph::make_ring()
--------	---------------------

network	-
---------	---

CREATE A STAR NETWORK

snafun	-
--------	---

igraph	igraph::make_star()
--------	---------------------

network	-
---------	---

CREATE A RANDOM GRAPH WITH GIVEN DENSITY

snafun	<pre>snafun::create_random_graph(n_vertices = 10, strategy = "gnm", # to fix the number of edges m = 27, # number of edges required, resulting density = .3 directed = TRUE, # or FALSE graph = c("igraph", "network")) # pick one snafun::create_random_graph(n_vertices = 10,</pre>
--------	--

¹ The output is a matrix object, not a 'network' object.

² This function is quite unintuitive and cumbersome

Graph creation

```
strategy = "gnp", # to fix the probability edges
p = .3, # probability for each edge, yields approx. density of .3
directed = TRUE, # or FALSE
graph = c("igraph", "network"))
```

```
igraph::sample_gnm()
igraph::sample_gnp()

igraph
igraph::make_directed_graph(n = 10, edges = 27)
igraph::make_undirected_graph(n = 10, edges = 27)
```

```
# 10 vertices, density on average .3
network1 sna::rgraph(n = 10, m = 1, tprob = 0.30, mode = 'digraph')

# 10 vertices, 27 edges (ie. density = .3)
sna::rgnm(1, 10, m = 27)
```

CREATE A RANDOM GRAPH WITH GIVEN DYAD CENSUS

```
snafun create_census_graph(n_vertices = 10, mut = 8, asym = 25, null = 12,
                           method = 'exact',
                           graph = 'igraph')
```

```
igraph -
```

```
network1 # 5 networks, each with 10 vertices, and 8 M, 25 A, and 12 N dyads
sna::rguman(n = 5, nv = 10, mut = 8, asym = 25, null = 12,
            method = "exact")
```

CREATE A RANDOM GRAPH WITH GIVEN COMMUNITY STRUTURE

```
snafun::create_community_graph(
  communitySizes = c(10, 20, 30),
  p_intra = c(0.3, 0.2, 0.3),
  p_inter = 0.2,
  p_del = 0,
  graph = 'igraph')
```

```
igraph -
```

```
network -
```

CREATE A GRAPH WITH GIVEN COMPONENTS

```
snafun snafun::create_components_graph(
```

¹ The output is a matrix object, not a 'network' object.

² This function is quite unintuitive and cumbersome

Graph creation

```
n_vertices,
directed = FALSE,
membership = NULL,
graph = 'igraph')
```

```
igraph      -
```

```
network     -
```

CREATE RANDOM BIPARTITE GRAPH

```
snafun      snafun::create_bipartite(
              n_type1,                # number of vertices of type 1
              n_type2,                # number of vertices of type 2
              strategy = c("gnp", "gnm"),
              p,                      # probability of each cross-type edge
              m,                      # number of cross-type edges
              directed = FALSE,
              mode = c("out", "in", "all"),
              graph = c("igraph", "network")
            )
```

```
igraph      igraph::sample_bipartite()
```

```
network2    network::network.bipartite()
```

CREATE GRAPH OBJECT FROM INPUT DATA

```
snafun      # `x` can be:
              # - an edgelist (in data.frame format)
              # - an incidence matrix (in matrix format)--for a bipartite graph
              # - an adjacency matrix (in matrix format)
              # - `vertices` can be a data.frame containing vertex attributes

              snafun::to_igraph(x, bipartite = FALSE, vertices = NULL)

              snafun::to_network(x, bipartite = FALSE, vertices = NULL)
```

```
igraph      # from an adjacency matrix
              igraph::graph_from_adjacency_matrix(adjmatrix,
              mode = c("directed", "undirected", "max", "min", "upper", "lower", "plus"),
              weighted = NULL, diag = TRUE, add.colnames = NULL, add.rownames = NA)

              igraph::make_graph(edges, ..., n = max(edges), isolates = NULL, directed = TRUE,
              dir = directed, simplify = TRUE)
```

¹ The output is a matrix object, not a 'network' object.

² This function is quite unintuitive and cumbersome

Graph creation

```
# from an adjacency list
igraph::graph_from_adj_list(adjlist, mode = c("out", "in", "all", "total"),
  duplicate = TRUE)

# from an edgelist
igraph::graph_from_edgelist(el, directed = TRUE)

# from a data.frame
igraph::graph_from_data_frame(d, directed = TRUE, vertices = NULL)

# from an incidence matrix (in matrix format)
igraph::graph_from_incidence_matrix(incidence, directed = FALSE,
  mode = c("all", "out", "in", "total"), multiple = FALSE,
  weighted = NULL, add.names = NULL)
```

```
# x can be an adjacency matrix (as a matrix), an incidence matrix (as a matrix),
# or an edgelist (as a data.frame)
network::network(x, vertex.attr = NULL, vertex.attrnames = NULL, directed = TRUE,
  hyper = FALSE, loops = FALSE, multiple = FALSE, bipartite = FALSE, ...)

# if x is a data.frame
network::as.network(x, directed = TRUE, vertices = NULL, hyper = FALSE,
  loops = FALSE, multiple = FALSE, bipartite = FALSE,
  bipartite_col = "is_actor", ...)

# if x is a matrix
network::as.network(x, matrix.type = NULL, directed = TRUE, hyper = FALSE,
  loops = FALSE, multiple = FALSE, bipartite = FALSE, ignore.eval = TRUE,
  names.eval = NULL, na.rm = FALSE, edge.check = FALSE, ...)
```

network

¹ The output is a matrix object, not a 'network' object.

² This function is quite unintuitive and cumbersome

Additional graph creation functions

After creating a graph, you can also extend it with helper functions such as `snafun::add_vertices()` and `snafun::add_edges()` .

Converting between graph classes

Graph object conversion

CONVERT TO AN ADJACENCY MATRIX

igraph	igraph::as_adjacency_matrix(g, sparse = FALSE)
--------	--

network	network::as.sociomatrix(g) network::as.matrix.network(g)
---------	---

CONVERT TO AN ADJACENCY LIST

igraph	igraph::as_adj_list(g)
--------	------------------------

network	-
---------	---

CONVERT TO AN EDGE LIST

igraph	igraph::as_edgelist(g) igraph::as_data_frame(g)
--------	--

network	network::as.data.frame.network(g) network::as.edgelist(g) # if you want to include edge weights sna::as.edgelist.sna(g)
---------	--

MAKE THE NETWORK DIRECTED

snafun	-
--------	---

igraph	igraph::as.directed(g)
--------	------------------------

network	-
---------	---

MAKE THE NETWORK UNDIRECTED

snafun	# from an adjacency matrix as input snafun::to_symmetric_matrix(g, rule = c("weak", "mutual", "out", "in", "average", "max", "min"), na.rm = TRUE)
--------	---

igraph	igraph::as.undirected(g) # for example
--------	---

Graph object conversion	
	<code>igraph::as_undirected(g, mode = 'collapse', edge.attr.comb = list(weight = 'sum'))</code>
network	<pre># from an adjacency matrix as input sna::symmetrize(g) # for example sna::symmetrize(g, rule = "strong")</pre>

PROJECT A BIPARTITE GRAPH

snafun	<pre>snafun::make_projection(x, which = "both") # or request a single partition snafun::make_projection(x, which = "row") snafun::make_projection(x, which = "column")</pre>
igraph	<code>igraph::bipartite_projection(g)</code>
network	—

CONVERT TO A LINE GRAPH

snafun	—
igraph	<code>igraph::make_line_graph(g)</code>
network	—

Additional graph creation functions

The cheatsheet below focuses on the most common creation routes. The package also contains helper functions for adding vertices and edges after creation.

Convert between various formats Using the consistent functions of snafun				
		OUTPUT		
INPUT	edgelist	matrix	igraph	network
edgelist	<code>to_edgelist()</code>	<code>to_matrix()</code>	<code>to_igraph()</code>	<code>to_network()</code>
matrix	<code>to_edgelist()</code>	<code>to_matrix()</code>	<code>to_igraph()</code>	<code>to_network()</code>
igraph	<code>to_edgelist()</code>	<code>to_matrix()</code>	<code>to_igraph()</code>	<code>to_network()</code>
network	<code>to_edgelist()</code>	<code>to_matrix()</code>	<code>to_igraph()</code>	<code>to_network()</code>

Conversion includes bipartite graphs.
Check the arguments for the various options.

Manipulating the graph object

Manipulate the graph object

PRINT THE GRAPH OBJECT

snafun	print(x)
--------	----------

igraph ¹	igraph::print.igraph(x)
---------------------	-------------------------

network ¹	network::print.network(x)
----------------------	---------------------------

CHECK CHARACTERISTICS OF THE GRAPH OBJECT

snafun	snafun::has_edge_attributes(x)
	snafun::has_vertex_attributes(x)
	snafun::has_vertex_attribute(x, attrname)
	snafun::has_edge_attribute(x, attrname)
	snafun::has_vertexnames(x)
	snafun::has_loops(x)
	snafun::is_bipartite(x)
	snafun::is_connected(x, rule = c("weak", "strong"))
	snafun::is_directed(x)
	snafun::is_network(x)
snafun::is_igraph(x)	
snafun::is_signed(x)	
snafun::is_weighted(x)	

igraph	# various functions, including igraph::any_loop(g) igraph::any_multiple(g) igraph::is_bipartite(graph) igraph::is_connected(graph, mode = c("weak", "strong")) igraph::is_directed(graph) igraph::is_igraph(graph)
--------	--

¹ Often, but certainly not always, it will also work if you just use `print(x)`

² These functions have many additional arguments

Manipulate the graph object

```
igraph::is_named(graph)
igraph::is_simple(graph)
igraph::is_weighted(graph)
```

network

```
# various functions, including:
network::has_loops(x)
network::is_bipartite(x)
network::is_directed(x)
network::is_multiplex(x)
network::is_network(x)
sna::is_connected(g, connected = "strong")
```

ACCESS THE VERTICES AND EDGES

snafun

—

igraph

```
igraph::V(x)      # vertices
igraph::E(x)      # edges
```

network

—

EXTRACT VERTEX / EDGE / GRAPH ATTRIBUTES

snafun

```
snafun::extract_vertex_attribute(x, name)
snafun::extract_vertex_names(x)          # specific to access the vertex names attribute
snafun::extract_edge_attribute(x, name)
snafun::extract_graph_attribute(x, name)
```

igraph

```
igraph::vertex_attr(graph, name, index = V(graph))
igraph::V(graph)$attributename
```

```
igraph::edge_attr(graph, name, index = E(graph))
igraph::E(graph)$attributename
```

```
igraph::graph_attr(graph, name)
```

network²

```
network::get.vertex.attribute(x, attrname)
network::get.edge.attribute(x, attrname)
network::get.network.attribute(x, attrname)
```

igraph

—

network

—

¹ Often, but certainly not always, it will also work if you just use `print(x)`

² These functions have many additional arguments

Manipulate the graph object

EXTRACT THE ID OF AN EDGE

snafun	snafun::extract_edge_id(object, ego, alter, edgelist)
--------	---

EXTRACT VERTEX NAMES

snafun	snafun::extract_vertex_names(x)
--------	---------------------------------

igraph	igraph::V(x)\$name igraph::vertex_attr(x, name = "names")
--------	--

network	network::get.vertex.attribute(x, "vertex.names") network::network.vertex.names(x)
---------	--

SET VERTEX NAMES

snafun	snafun::add_vertex_names(x, value)
--------	------------------------------------

igraph	igraph::V(x)\$name <- value igraph::set_vertex_attr(x, name = "names")
--------	--

network	network::set.vertex.attr(x, "vertex.names", value) network::network.vertex.names(x) <- value
---------	--

LIST VERTEX / EDGE / GRAPH ATTRIBUTES

snafun	snafun::list_vertex_attribute(x, name) snafun::list_edge_attribute(x, name) snafun::list_graph_attribute(x, name)
--------	---

igraph	igraph::vertex_attr_names(graph) igraph::edge_attr_names(graph) igraph::graph_attr_names(graph)
--------	---

network	network::list.vertex.attributes(x) network::list.edge.attributes(x)
---------	--

¹ Often, but certainly not always, it will also work if you just use `print(x)`

² These functions have many additional arguments

Manipulate the graph object	
network::list.network.attributes(x)	

EXTRACT ALL VERTEX ATTRIBUTES INTO A DATA.FRAME

snafun	snafun::extract_all_vertex_attributes(g)
igraph	–
network	–

ADD VERTEX / EDGE / GRAPH ATTRIBUTES

snafun	# very flexible and powerful functions to add attributes # in several ways, with the same function snafun::add_edge_attributes(object, attr_name, value, edgelist, overwrite = FALSE) snafun::add_vertex_attributes(x, attr_name = NULL, value) snafun::add_graph_attribute(x, attr_name = NULL, value)
igraph	igraph::set_edge_attr(g, "name", value) igraph::E(g)\$name &lt;- value igraph::set_vertex_attr(g, "name", value) igraph::V(g)\$name &lt;- value igraph::set_graph_attr(g, "name", value) g\$name &lt;- value
network	network::set.edge.attribute(g, "new_name", value, e = seq_along(g\$mel)) network::set.edge.value(g, "new_name", value, e = seq_along(g\$mel)) network::set.network.attribute(g, "new_name", value) network::set.vertex.attribute(g, "new_name", value, v = seq_len(network::network.size(g)))

REMOVE VERTEX / EDGE / GRAPH ATTRIBUTES

snafun	snafun::remove_edge_attribute(x, attr_name) snafun::remove_edge_weight(x) # remove edge weights snafun::remove_vertex_attribute(x, attr_name) snafun::remove_vertex_names(x) # specific for the vertex name attribute snafun::remove_graph_attribute(x, attr_name)
--------	---

¹ Often, but certainly not always, it will also work if you just use `print(x)`
² These functions have many additional arguments

Manipulate the graph object

igraph	igraph::delete_edge_attr(graph, name) igraph::delete_vertex_attr(graph, name) igraph::delete_graph_attr(graph, name)
network	network::delete.edge.attribute(x, attrname, ...) network::delete.vertex.attribute(x, attrname, ...) network::delete.network.attribute(x, attrname, ...)

ADD VERTICES OR EDGES

snafun	snafun::add_vertices(x, vertices) snafun::add_edges(x, edges)
igraph	igraph::add_vertices(graph, nv, ..., attr = list()) igraph::add_edges(graph, edges, ..., attr = list())
network	network::add.vertices(x, nv, vattr = NULL, last.mode = TRUE, ...) network::add.edge(x, tail, head, names.eval = NULL, vals.eval = NULL, edge.check = FALSE, ...) network::add.edges(x, tail, head, names.eval = NULL, vals.eval = NULL, ...)

REMOVE VERTICES OR EDGES

snafun	snafun::remove_vertices(x, vertices)
igraph	igraph::delete_vertices(graph, v) igraph::delete_edges(graph, edges)
network	network::delete.edges(x, eid) network::delete.vertices(x, vid)

CONTRACT VERTICES INTO ONE

snafun	snafun::contract_vertices(g, vertices, method = c("min", "max", "union", "add"), attr = NULL, new_name = "set", out_as_igraph = TRUE)
igraph	—
network	—

¹ Often, but certainly not always, it will also work if you just use `print(x)`

² These functions have many additional arguments

Manipulate the graph object

EXTRACT A SUBSET FROM THE GRAPH

snafun	# single function to do it all snafun::extract_subgraph(x, v_to_keep, e_to_keep)
igraph	# subset based on vertices igraph::induced_subgraph(g, vids = theVerticesYouWantToKeep) # subset based on edges igraph::subgraph.edges(g, eids = theEdgesYouWantToKeep)
network	# subset based on vertices network::get.inducedSubgraph(g, v = theVerticesYouWantToKeep) # subset based on edges network::get.inducedSubgraph(g, eid = theEdgesYouWantToKeep)

EXTRACT EGONETS FROM THE GRAPH

snafun	snafun::extract_ego(x, vertices = NULL, order = 1, type = c("all", "out", "in"))
igraph	igraph::make_ego_graph(g, order = 1, nodes = V(graph), mode = c("all", "out", "in"), mindist = 0)
network	sna::ego.extract(dat, ego = NULL, neighborhood = c("combined", "in", "out"))

CLEAN UP THE GRAPH

snafun	snafun::find_isolates(x, names = TRUE, loops = FALSE) snafun::remove_isolates(x, loops = FALSE) snafun::remove_loops(x) snafun::has_multiple_edges(x) snafun::extract_multiple_edges(x) snafun::remove_multiple_edges(x)
igraph	igraph::simplify(g) # for example igraph::simplify(g, remove.multiple = TRUE, remove.loops = TRUE, edge.attr.comb = list(weight = 'max'))

¹ Often, but certainly not always, it will also work if you just use `print(x)`

² These functions have many additional arguments

Manipulate the graph object

network sna::isolates(g, diag=FALSE)

UNION OF GRAPHS

snafun snafun::make_union(net1, net2, net3, byname = 'auto')

igraph igraph::union(net1, net2, net3)

network –

INTERSECTION OF GRAPHS

snafun snafun::extract_intersection(net, net2, net3, byname = 'auto', keep.all.vertices = TRUE)

igraph igraph::intersection(net, net2, net3)

network –

¹ Often, but certainly not always, it will also work if you just use `print(x)`

² These functions have many additional arguments

Graph level indices

Explore the graph

SUMMARIZE THE NETWORK

snafun	snafun::g_summary(x, directed = TRUE)
--------	---------------------------------------

igraph	—
--------	---

network	—
---------	---

COUNT THE NUMBER OF VERTICES

snafun	snafun::count_vertices(x)
--------	---------------------------

igraph	igraph::vcount()
--------	------------------

network	network::network.size()
---------	-------------------------

COUNT THE NUMBER OF EDGES

snafun	snafun::count_edges(x)
--------	------------------------

igraph	igraph::ecount(x) igraph::gsize(x)
--------	---------------------------------------

network	network::network.edgecount(x)
---------	-------------------------------

DENSITY

snafun	snafun::g_density(x, loops = FALSE)
--------	-------------------------------------

igraph	igraph::edge_density(g)
--------	-------------------------

network	# preferable for bipartite graphs network::network_density(g) # preferable for valued graphs
---------	--

¹ igraph and sna only calculate 'Freeman' centralization (snafun does both 'Freeman' and 'sd')

² '\$res' or '\$vector' return the centrality scores

Explore the graph	
	sna::gden(g)

RECIPROCITY

snafun	snafun::g_reciprocity(x)
igraph	igraph::reciprocity(g)
network	sna::grecip(g, 'measure = 'edgewise')

TRANSITIVITY

snafun	snafun::g_transitivity(x)
igraph	igraph::transitivity(g, type = 'global')
network	sna::gtrans(g, mode = 'digraph', measure = 'weak', use.adjacency = TRUE)

MEAN DISTANCE BETWEEN VERTICES

snafun	snafun::g_mean_distance(x)
igraph	igraph::mean_distance(g, directed = TRUE, unconnected = TRUE)
network	–

DEGREE DISTRIBUTION

snafun	snafun::g_degree_distribution(x, mode = c("out", "in", "all"), type = c("density", "count"), cumulative = FALSE, loops = FALSE, digits = 3)
igraph	igraph::degree_distribution(graph, cumulative = FALSE, mode = 'out')
network	–

¹ igraph and sna only calculate `Freeman` centralization (snafun does both `Freeman` and `sd`)

² ``$res`` or ``$vector`` return the centrality scores

Explore the graph

DYAD CENSUS

snafun	snafun::count_dyads(x, echo = TRUE)
igraph	igraph::dyad_census(g)
network	sna::dyad.census(g)

TRIAD CENSUS

snafun	snafun::count_triads(x, echo = TRUE)
igraph	igraph::triad_census(g)
network	sna::triad.census(g, mode = 'digraph')

DEGREE ASSORTATIVITY

snafun	–
igraph	igraph::assortativity_degree(g, directed = TRUE)
network	–

DIAMETER

snafun	snafun::g_diameter(x, directed = is_directed(x), unconnected = TRUE)
igraph	igraph::diameter(g, directed = TRUE, unconnected = TRUE) # what is the vertex pair with the longest geodesic igraph::farthest_vertices(g)
network	–

RADIUS

snafun	snafun::g_radius(x, mode = c("all", "out", "in"))
--------	---

¹ igraph and sna only calculate `Freeman` centralization (snafun does both `Freeman` and `sd`)
² `res` or `vector` return the centrality scores

Explore the graph

igraph	igraph::radius(graph, mode = c("all", "out", "in", "total"))
network	—

COMPACTNESS

snafun	snafun::g_compactness(x, mode = c("out", "in", "all"))
igraph	—
network	—

CENTRALIZATION

	<pre># a single function to calculate centralization (either `Freeman` or `sd`) # on a wide range of centrality indices</pre>
snafun	<pre>snafun::g_centralize(x, measure = "betweenness", directed = TRUE, mode = c("all", "out", "in"), k = 3, damping = 0.85, normalized = TRUE, method = c("freeman", "sd"))</pre>
	<pre># general function for Freeman centralization igraph::centralize(scores, theoretical.max = 0, normalized = TRUE)</pre>
	<pre># betweenness centralization igraph::centr_betw(g, directed = TRUE)</pre>
igraph ^{1,2}	<pre># closeness centralization igraph::centr_clo(g, mode = 'out', normalized = FALSE) # degree centralization igraph::centr_degree(g, mode = 'all')</pre>
	<pre># eigenvector centralization igraph::centr_eigen(g, directed = TRUE)</pre>
network ¹	<pre># general function for Freeman centralization, # include any function that calculates vertex centrality sna::centralization(g, FUN, mode = 'digraph', normalize=TRUE, ...)</pre>

MIXING MATRIX

snafun	snafun::make_mixingmatrix(x, attrname, by_edge = FALSE, loops = has_loops(x))
--------	---

¹ igraph and sna only calculate `Freeman` centralization (snafun does both `Freeman` and `sd`)
² ``$res`` or ``$vector`` return the centrality scores

Explore the graph

igraph	–
network	network::mixingmatrix(object, attrname, useNA = "ifany", expand.bipartite = FALSE)

CORRELATION BETWEEN TWO GRAPHS

snafun	snafun::g_correlation(g1, g2, diag = FALSE)
igraph	–
network	sna::gcor(g1, g2, mode = 'graph')

FAST-GREEDY COMMUNITY DETECTION

snafun	snafun::extract_comm_fastgreedy(x, weights = NA, modularity = TRUE, merges = TRUE, membership = TRUE)
igraph	igraph::cluster_fast_greedy(g, merges = TRUE, modularity = TRUE, membership = TRUE, weights = NULL)
network	–

GIRVAN-NEWMAN COMMUNITY DETECTION

snafun	snafun::extract_comm_girvan(x, weights = NA, directed = TRUE, modularity = TRUE, edge.betweenness = FALSE, bridges = FALSE, merges = TRUE, membership = TRUE)
igraph	igraph::cluster_edge_betweenness(g, weights = NULL, directed = TRUE, edge.betweenness = TRUE, merges = TRUE, bridges = TRUE, modularity = TRUE, membership = TRUE)
network	–

LOUVAIN COMMUNITY DETECTION

snafun	snafun::extract_comm_louvain(x, weights = NA, resolution = 1)
igraph	igraph::cluster_louvain(graph, weights = NULL, resolution = 1)
network	–

WALKTRAP COMMUNITY DETECTION

¹ igraph and sna only calculate `Freeman` centralization (snafun does both `Freeman` and `sd`)
² ``$res`` or ``$vector`` return the centrality scores

Explore the graph	
snafun	snafun::extract_comm_walktrap(x, weights = NA, steps = 4, modularity = TRUE, merges = TRUE, membership = TRUE)
igraph	igraph::cluster_walktrap(g, weights = NULL, steps = 4, merges = TRUE, modularity = TRUE, membership = TRUE)
network	—

MERGE COMMUNITY MEMBERSHIP

snafun	snafun::merge_membership(coms, merges)
igraph	—
network	—

COMMUNITY MEMBERSHIP

snafun	snafun::extract_comm_membership(coms)
igraph	igraph::membership(coms)
network	—

NUMBER OF COMMUNITIES

snafun	snafun::count_communities(coms)
igraph	length(coms)
network	—

COMMUNITY SIZES

snafun	snafun::extract_comm_sizes(coms)
igraph	igraph::sizes(coms)
network	—

COMMUNITY MODULARITY

¹ igraph and sna only calculate `Freeman` centralization (snafun does both `Freeman` and `sd`)

² ``\$res` or ``\$vector` return the centrality scores

Explore the graph

snafun	snafun::extract_comm_modularity(coms)
--------	---------------------------------------

igraph	igraph::modularity(coms)
--------	--------------------------

network	—
---------	---

COMMUNITY CROSSING TIES

snafun	snafun::extract_comm_crossing(coms, graph)
--------	--

igraph	igraph::crossing(coms, graph)
--------	-------------------------------

network	—
---------	---

EVALUATE COMMUNITIES

snafun	snafun::evaluate_communities(coms, graph, sil_width = FALSE)
--------	--

igraph	—
--------	---

network	—
---------	---

PLOT SILHOUETTE WIDTHS FOR COMMUNITIES

snafun	evls <- snafun::evaluate_communities(coms, graph) plot(evls)
--------	---

igraph	—
--------	---

network	—
---------	---

PLOT COMMUNITIES

snafun	snafun::plot_communities(coms, graph)
--------	---------------------------------------

igraph	plot(coms, graph)
--------	-------------------

network	—
---------	---

ADD COMMUNITY MEMBERSHIP TO GRAPH

snafun	snafun::add_comm_membership(graph, coms, attr = "community")
--------	--

¹ igraph and sna only calculate `Freeman` centralization (snafun does both `Freeman` and `sd`)
² ``$res`` or ``$vector`` return the centrality scores

Explore the graph	
igraph	—
network	—
ADD COMMUNITY CROSSING TO GRAPH	
snafun	snafun::add_comm_crossing(graph, coms, attr = "crossing")
igraph	—
network	—

1 igraph and sna only calculate `Freeman` centralization (snafun does both `Freeman` and `sd`)
2 ``$res`` or ``$vector`` return the centrality scores

Communities and other subgroups

Community results returned by `snafun::extract_comm_*`() are compatible with the wider `igraph` community API. You can inspect memberships, sizes, crossings, modularity, and evaluate the detected communities with `snafun::evaluate_communities()` .

Vertex-level indices

Explore the vertices	
DEGREE	
snafun	snafun::v_degree(x, vids = NULL, mode = c("all", "out", "in"), loops = FALSE, rescaled = FALSE)
igraph	igraph::degree(g, mode = 'out')
network	sna::degree(g, gmode = 'digraph', cmode = 'outdegree')
BETWEENNESS	
snafun	snafun::v_betweenness(x, vids = NULL, directed = TRUE, rescaled = FALSE)
igraph	igraph::betweenness(g, directed = TRUE)
network	sna::betweenness(g, gmode = 'digraph', cmode = 'directed')
FLOW BETWEENNESS	
snafun	—
igraph	—
network	sna::flowbet(g, gmode = 'digraph', cmode = 'rawflow')
BONACICH POWER CENTRALITY	
snafun	—
igraph	igraph::power_centrality(g)
network	sna::bonpow(g, gmode = 'digraph')
CLOSENESS CENTRALITY	
snafun	snafun::v_closeness(x, vids = NULL, mode = c("all", "out", "in"), rescaled = FALSE)

¹ These bridge measures need a community assignment. `communities` can be the output of `snafun::extract\_comm\_\*()`, a membership vector, or a named list of communities.

² For these shortest-path bridge measures, positive weights are translated to distances via `1/weight`. Zero and negative weights are ignored in the path search; use `weights = NA` for purely topological paths.

³ These functions serve equivalent purposes, but yield quite different kinds of outputs

Explore the vertices

igraph	<code>igraph::closeness(g, mode = 'all')</code>
network	<code>sna::closeness(g, gmode = 'digraph', cmode = 'directed')</code>

BRIDGE STRENGTH

1 snafun	<code>snafun::v_bridge_strength(x, communities, type = c("all", "out", "in"), absolute = TRUE, rescaled = FALSE)</code>
igraph	—
network	—

BRIDGE EXPECTED INFLUENCE

1 snafun	<code>snafun::v_bridge_expected_influence(x, communities, type = c("all", "out", "in"), rescaled = FALSE)</code>
igraph	—
network	—

BRIDGE EXPECTED INFLUENCE (2-STEP)

1 snafun	<code>snafun::v_bridge_expected_influence2(x, communities, type = c("all", "out", "in"), rescaled = FALSE)</code>
igraph	—
network	—

BRIDGE CLOSENESS

¹ These bridge measures need a community assignment. `communities` can be the output of `snafun::extract\_comm\_\*()`, a membership vector, or a named list of communities.

² For these shortest-path bridge measures, positive weights are translated to distances via `1/weight`. Zero and negative weights are ignored in the path search; use `weights = NA` for purely topological paths.

³ These functions serve equivalent purposes, but yield quite different kinds of outputs

Explore the vertices

snafun ²	snafun::v_bridge_closeness(x, communities, mode = c("out", "in", "all"), weights = NULL, rescaled = FALSE)
igraph	–
network	–

BRIDGE BETWEENNESS

snafun ²	snafun::v_bridge_betweenness(x, communities, directed = NULL, weights = NULL, rescaled = FALSE)
igraph	–
network	–

HARMONIC CENTRALITY

snafun	snafun::v_harmonic(x, vids = NULL, mode = c("all", "out", "in"), rescaled = FALSE)
igraph	igraph::harmonic centrality(g, vids = V(graph), mode = c("out", "in", "all"), weights = NULL)
network	–

STRESS CENTRALITY

snafun	snafun::v_stress(x, vids = NULL, directed = TRUE, rescaled = FALSE)
igraph	–
network	sna::stresscent(g, gmode = 'digraph', cmode = 'directed')

ECCENTRICITY

¹ These bridge measures need a community assignment. `communities` can be the output of `snafun::extract\_comm\_\*()` , a membership vector, or a named list of communities.

² For these shortest-path bridge measures, positive weights are translated to distances via `1/weight` . Zero and negative weights are ignored in the path search; use `weights = NA` for purely topological paths.

³ These functions serve equivalent purposes, but yield quite different kinds of outputs

Explore the vertices

snafun	snafun::v_eccentricity(x, vids = NULL, mode = c("all", "out", "in"), rescaled = FALSE)
igraph	igraph::eccentricity(g, mode = 'all')
network	–

EIGENVECTOR

snafun	snafun::v_eigenvector(x, directed = TRUE, rescaled = FALSE)
igraph	igraph::eigen_centrality(g, directed = TRUE, scale = FALSE)\$vector
network	sna::evcent(g, gmode = 'digraph', rescale=FALSE)

PAGE RANK

snafun	snafun::v_pagerank(x, vids = NULL, damping = 0.85, directed = TRUE, rescaled = FALSE)
igraph	igraph::page_rank(g, vids = V(graph), directed = TRUE, damping = 0.85, weights = NULL)
network	–

GEO K-PATH

snafun	<pre># without using weights snafun::v_geokpath(x, vids = NULL, mode = c("all", "out", "in"), k = 3, rescaled = FALSE) # if weights are to be used snafun::v_geokpath_w(x, vids = NULL, mode = c("all", "out", "in"), weights = NULL, k = 3)</pre>
igraph	–
network	–

SHAPLEY CENTRALITY

snafun	snafun::v_shapley(x, add.vertex.names = FALSE, vids = NULL, rescaled = FALSE)
--------	---

¹ These bridge measures need a community assignment. `communities` can be the output of `snafun::extract\_comm\_\*()`, a membership vector, or a named list of communities.

² For these shortest-path bridge measures, positive weights are translated to distances via `1/weight`. Zero and negative weights are ignored in the path search; use `weights = NA` for purely topological paths.

³ These functions serve equivalent purposes, but yield quite different kinds of outputs

Explore the vertices	
igraph	–
network	–

WHO ARE THE NEIGHBORS OF A VERTEX

snafun	<code>snafun::extract_neighbors(x, vertex, type = c("out", "in", "all"))</code>
igraph	<code>igraph::neighbors(g, 'Jane', mode = 'out')</code> <code># all options</code> <code>igraph::neighbors(graph, v, mode = c('out', 'in', 'all', 'total'))</code>
network	<code>network::get.neighborhood(g, 1, 'out')</code> <code># all options</code> <code>network::get.neighborhood(x, v, type = c('out', 'in', 'combined'), na.omit = TRUE)</code>

NEIGHBORHOOD OF A VERTEX³

snafun	–
igraph	<code>igraph::make_ego_graph(g, order = 1, nodes = "Jane", mode = "all")</code> <code># all options</code> <code>igraph::make_ego_graph(graph, order = 1, nodes = V(graph),</code> <code> mode = c("all", "out", "in"), mindist = 0)</code>
network	<code>sna::ego.extract(dat, ego = NULL, neighborhood = c("combined", "in", "out"))</code> <code>sna::neighborhood(dat, order, neighborhood.type = c("in", "out", "total"),</code> <code> mode = "digraph", diag = FALSE, thresh = 0, return.all = FALSE, partial = TRUE)</code>

¹ These bridge measures need a community assignment. `communities` can be the output of `snafun::extract\_comm\_\*()`, a membership vector, or a named list of communities.

² For these shortest-path bridge measures, positive weights are translated to distances via `1/weight`. Zero and negative weights are ignored in the path search; use `weights = NA` for purely topological paths.

³ These functions serve equivalent purposes, but yield quite different kinds of outputs

Dyad-level indices

Explore the dyads

SHORTEST PATH FOR A GIVEN SET OF VERTICES

snafun	–
igraph	<code>igraph::all_shortest_paths(g, from = IDofVertex, to = igraph::V(g), mode = 'out')</code>
network	–

GEODESIC LENGTHS¹

snafun	<code>snafun::d_distance(x, mode = c("all", "out", "in"))</code>
igraph	<code>igraph::distances(g, mode = 'out')</code>
network	<code>sna::geodist(g, count.paths = TRUE)\$counts</code>

STRUCTURAL EQUIVALENCE

snafun	<code>snafun::d_structural_equivalence(x, weights = NA, digits = 3, suppressWarnings = TRUE)</code>
igraph	–
network	<code>sna::sedist(g, method = "correlation", mode = "digraph", diag=FALSE)</code>

REGULAR EQUIVALENCE

snafun	<code>snafun::d_regular_equivalence(x, weights = NA, digits = 3)</code>
igraph	–
network	<code>1 - sna::redist(g, method = "catrege", mode = "digraph", diag = FALSE)</code>

EDGE BETWEENNESS

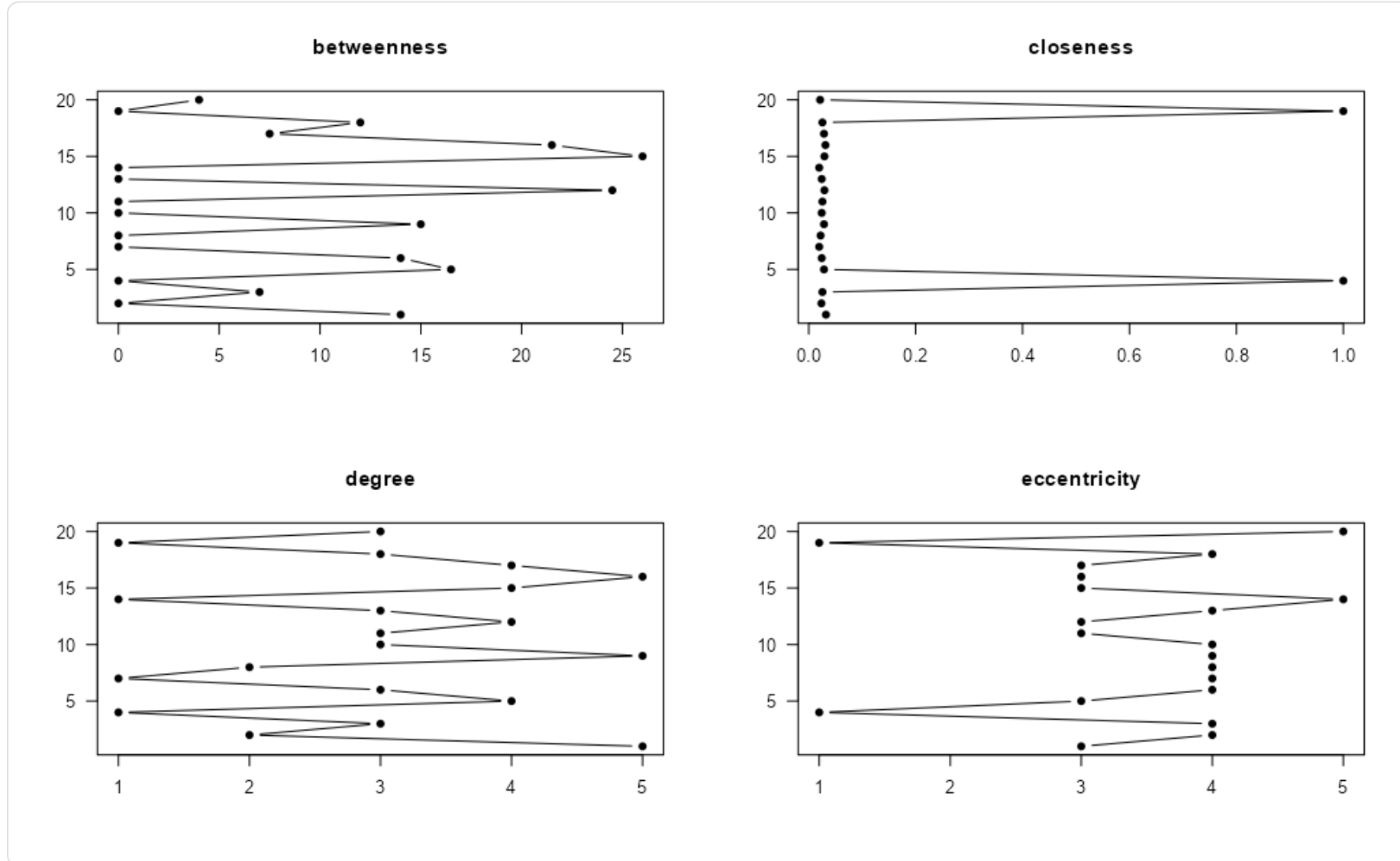
snafun	–
igraph	<code>igraph::edge.betweenness(g, directed = FALSE)</code>
network	–

¹ The output is a table with an entry per vertex pair

Plotting

Plotting in snafun

The snafun package provides S3 plotting methods, so a plain call to `plot(x)` gives a quick plot regardless of whether `x` is an `igraph` or `network` object. The package also includes a dedicated centrality comparison plot.



Example output from `snafun::plot_centralities()`.

Statistical models

Overview table

Statistical network models		
When	Which approach	Function
Dependent vertex attribute explained by a network weight matrix and a matrix of covariates	Network autocorrelation model	snafun::stat_nam
A statistic on a single network	Conditional Uniform Graph test	snafun::stat_cug
Association between two networks	QAP	snafun::stat_qap_cor
A valued dependent network explained by one or more explanatory networks	QAP linear model	snafun::stat_qap_lm
A binary dependent network explained by one or more explanatory networks	QAP logistic model	snafun::stat_qap_logit
A binary or valued dependent network explained by a set of endogenous and exogenous variables	Exponential random graph models	ergm::ergm

Network autocorrelation models

Use `snafun::stat_nam()` for network autocorrelation models with row-normalized weight matrices.

Conditional Uniform graphs (CUG)

Use `snafun::stat_cug()` for conditional uniform graph tests on supported graph classes.

QAP test

Use `snafun::stat_qap_cor()` to test the association between two networks.

QAP linear regression

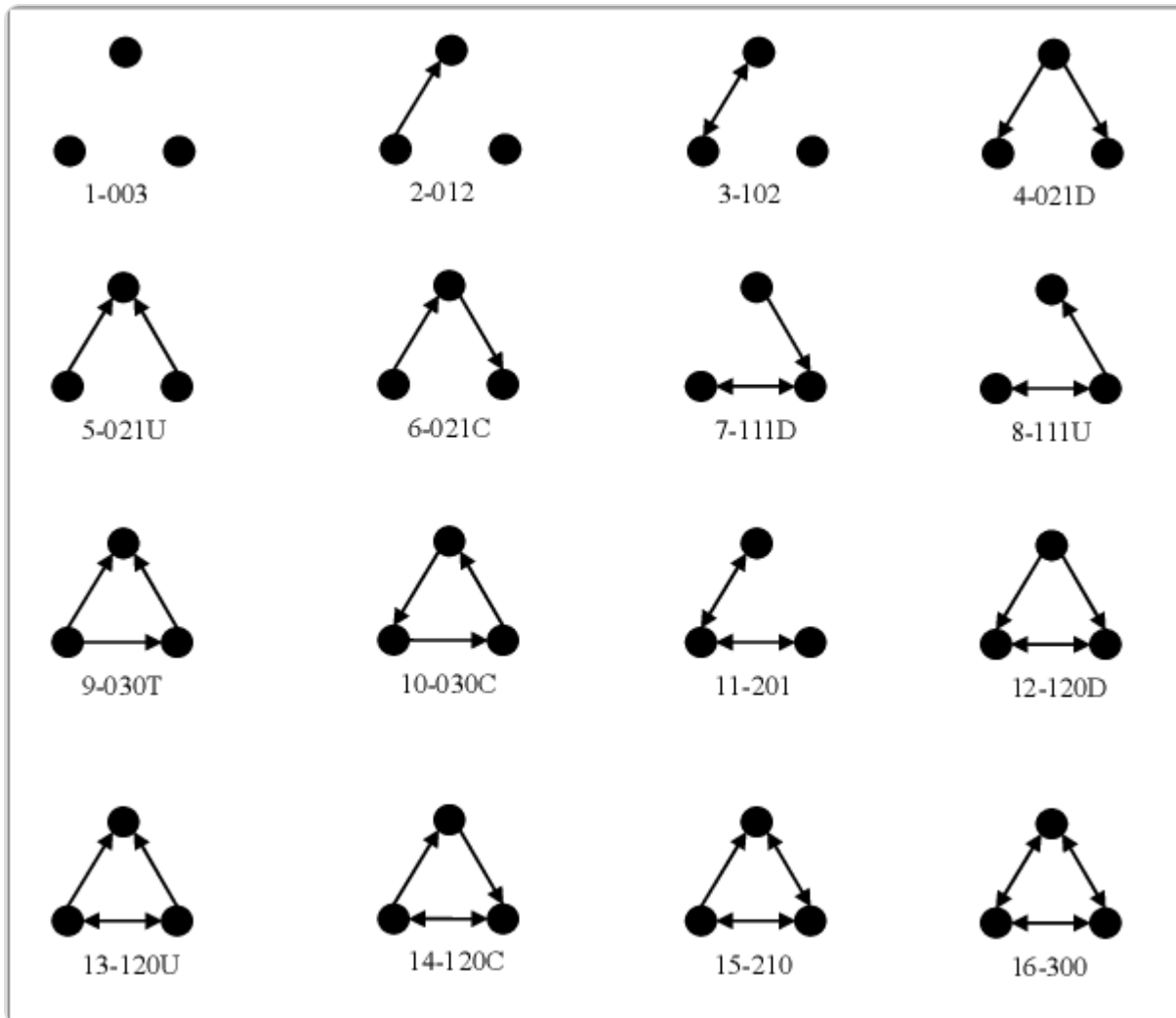
Use `snafun::stat_qap_lm()` for valued dependent networks explained by one or more predictor networks.

QAP logistic regression

Use `snafun::stat_qap_logit()` when the dependent network is binary.

Terms classification for every Exponential Random Graph Model (ERGM)

The static figures below are used in the course to explain common ERGM term families.

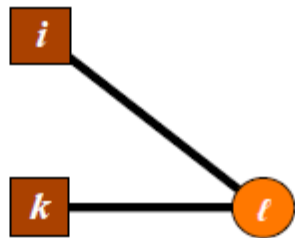
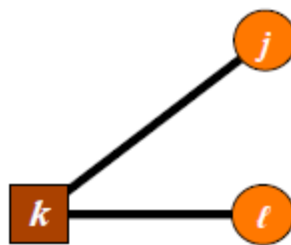
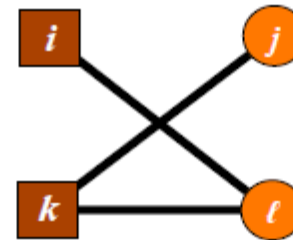


Triad census categories used in ERGM teaching.

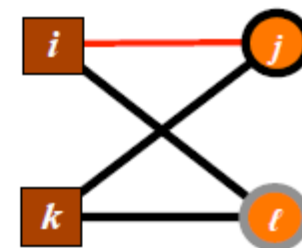
Bipartite ERGMs



Edges

"people"
2-stars"affiliation"
2-stars

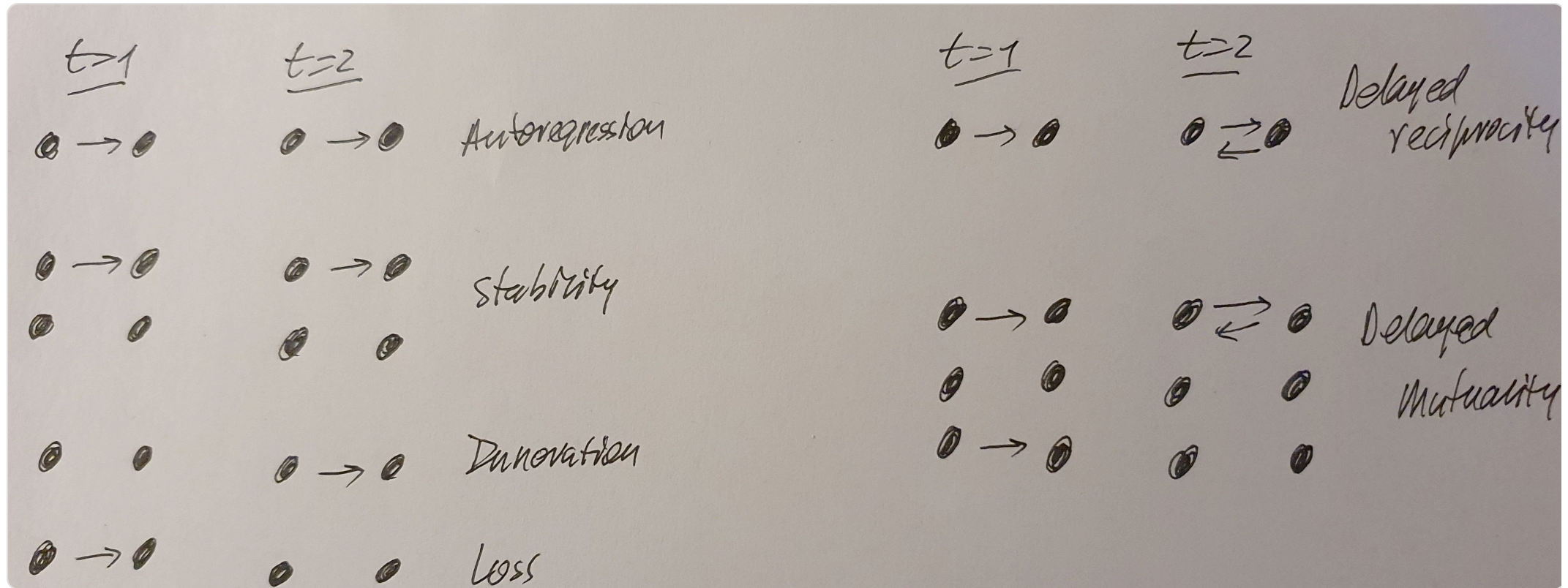
3-paths



4-cycles

Overview of common bipartite ERGM terms.

ERGM for temporal networks



Overview of common temporal ERGM terms.

Temporal effects for the ERGM		
	meaning	btergm
memory		
Positive autoregression	Previous existing edges persist in a next network	<code>btergm::memory(type = "autoregression", lag = 1)</code>
Dyadic stability	Both previous existing and non-existing ties are carried over to the current network	<code>btergm::memory(type = "stability", lag = 1)</code>
Edge innovation	A non-existing previous tie becomes existent in the current network	<code>btergm::memory(type = "innovation", lag = 1)</code>
Edge loss	An existing previous tie is dissolved in the current network	<code>btergm::memory(type = "loss", lag = 1)</code>
delayed reciprocity		
reciprocity	if node j is tied to node i at t = 1, does this lead to a reciprocation of that tie back from i to j at t = 2?	<code>btergm::delrecip(mutuality = FALSE, lag = 1)</code>
mutuality	if node j is tied to node i at t = 1, does this lead to a reciprocation of that tie back from i to j at t = 2 AND if i is not tied to j at t = 1, will this lead to j not being tied to i at t = 2? This captures a trend away from asymmetry.	<code>btergm::delrecip(mutuality = TRUE, lag = 1)</code>
time covariates		
time effect per se	Test for a specific trend (linear or non-linear) for edge formation	<code>btergm::timecov(transform = function(t) t)</code>
Time effect of a covariate	Interaction effect to test whether the importance of a covariate increases or decreases over time	<code>btergm::timecov(x, transform = function(t) t)</code>

Temporal networks (exploration and description)

Network generation and manipulation

In this course, temporal network work mostly relies on `networkDynamic` , `tsna` , and `ndtv` . The `snafun` API now also supports several active-attribute accessors for `networkDynamic` objects.

Network measures and descriptives

Temporal descriptives often focus on durations, edge formations, dissolutions, and time-varying ERGM terms.

Participation shifts

P-shift	Example
Turn receiving	
AB-BA	John talks to Mary, then Mary replies.
AB-B0	John talks to Mary, then Mary addresses the group.
AB-BY	John talks to Mary, then Mary talks to Irene.
Turn claiming	
A0-X0	John talks to the group, then Frank talks to the group.
A0-XA	John talks to the group, then Frank talks to John.
A0-XY	John talks to the group, then Frank talks to Mary.
Turn usurping	
AB-X0	John talks to Mary, then Frank talks to the group.
AB-XA	John talks to Mary, then Frank talks to John.
AB-XB	John talks to Mary, then Frank addresses Mary.
AB-XY	John talks to Mary, then Frank addresses Irene.
Turn continuing	
A0-AY	John talks to the group, then addresses Mary.
AB-A0	John talks to Mary, then makes a remark to the group.
AB-AY	John talks to Mary then to Irene.
<i>Note:</i> The initial speaker is always labeled A, and the initial target B, unless the group is addressed (or the target was ambiguous), in which case the target is 0. Then the shift is summarized in the form [speaker₁][target₁]-[speaker₂][target₂], with A or B appearing after the hyphen only if the initial speaker or target serves in one of these two positions after the shift. When the speaker after the shift is someone other than A or B, X is used, and when the target after the shift is someone other than A, B, or the group, Y is used.	

Participation-shift categories used in the temporal networks part of the course.